

MANUEL D'APPRENTISSAGE PAR LA PRATIQUE

SPRINT 3 — TÂCHE 01 : API RÔLES, PERMISSIONS & RATTACHEMENT

Guide d'ingénierie active pour programmer de vos propres mains les fonctions de création de rôles, d'attribution matricielle des permissions par module et de rattachement sécurisé d'un collaborateur.

Sprint : Sprint 3 (Gestion de Projets & RBAC)	Périmètre : apps/accounts
Rôle Développeur : Lead Backend Django / DRF	Niveau de Compétence : <i>Homo Docens</i>
Architecture : Multi-Tenant PostgreSQL (Schémas)	Contraintes : Zéro Copier-Coller • Saisie Manuelle

Cadrage Stratégique SCQA (Minto Pyramid)

Situation : Le système CCD Digital entre dans son Sprint 3 avec l'objectif de livrer une plateforme robuste de structuration et de pilotage de chantiers BTP. L'entreprise cliente (le tenant) a besoin d'une matrice de contrôle d'accès basée sur les rôles (RBAC) fine et dynamique.

Complication : Les rôles système de base (AD, DG, CP, CT, CC, etc.) ne suffisent pas à couvrir les organisations complexes du BTP (Chefs d'Équipe gros œuvre, Métreurs, Responsables QSE externes). De plus, rattacher un collaborateur à un rôle personnalisé ne doit jamais compromettre l'étanchéité multi-tenant, ni permettre l'usurpation du rôle souverain de Directeur Général / Fondateur.

Question : Comment programmer des fonctions de service atomiques et des endpoints REST impeccables permettant 1) de créer un rôle sur mesure, 2) de lui assigner des niveaux d'accès parmi 12 modules BTP, et 3) de rattacher ou réassigner un collaborateur tout en garantissant des invariants stricts de sécurité ?

Orientation : Ce manuel décompose pas-à-pas chaque fonction, chaque contrainte de modèle, chaque sérialiseur et chaque test pour que vous tapiez vous-même chaque caractère de code et développiez des réflexes d'ingénieur souverain.

Neuro-Pédagogie Active : Zéro Copier-Coller

Le Principe du Recyclage Neuronal & de l'Automatisation Moteur (Dehaene) :

L'IA ne doit jamais coder cette tâche à votre place. La compréhension passive est une illusion de compétence (Kahneman). En tapant chaque instruction Django, vous obligez votre cortex moteur et vos réseaux bayésiens à encoder la syntaxe, les transactions atomiques et les validations d'erreurs jusqu'à ce qu'elles deviennent des réflexes automatiques.

SECTION 1 : FILM MENTAL & OBJECTIF SACRÉ (JOSEPH MURPHY)

1.1 La Visualisation de la Réussite Finale (Murphy - Le Film Mental)

Fermez les yeux quelques instants avant d'ouvrir votre éditeur de code. Visualisez l'état final du système : le Directeur Général de l'entreprise BTP ouvre l'interface des paramètres. Il saisit un nouveau rôle '*Chef d'Équipe Coffrage*', coche '*Écriture*' sur le module **Chantier**, '*Lecture*' sur **QHSE**, et valide. En une fraction de seconde (moins de 40 ms), l'API DRF renvoie une réponse HTTP 201 Created avec la matrice complète JSON. Ensuite, il sélectionne un collaborateur dans sa liste et lui associe ce rôle. Immédiatement, les permissions de ce collaborateur sont recalculées sans faille. En console, la commande `pytest apps/accounts/tests/ -v` affiche 100% de pastilles vertes sans le moindre avertissement.

Auto-Suggestion & Conditionnement Mental

Affirmation d'Ancrage Subconscient :
« Mon esprit assimile avec calme et clarté les principes de l'architecture logicielle. Je comprends chaque relation entre l'Utilisateur, le Rôle et les Modules. Mon code est propre, performant, robuste et exempt de failles. Je progresse avec assurance vers l'état d'Homo Docens, capable d'enseigner ce que je maîtrise. »

1.2 La Loi de l'Effort Inversé & Gestion du Stress Cognitif

Le Dr. Joseph Murphy et les neurosciences cognitives modernes démontrent que plus vous vous crispez sur une difficulté ou une erreur de compilation, plus votre cerveau bloque l'accès à la mémoire de travail (le cortex préfrontal s'inhibe). Si un test échoue ou si une validation lève une exception inattendue :

- **Respirez profondément** : Ne forcez pas la solution dans la tension.
- **Accueillez le signal d'erreur** : Stanislas Dehaene enseigne que l'erreur n'est pas un échec, c'est le seul moteur d'ajustement bayésien du cerveau.
- **Décomposez le problème** : Isolez l'inconnu, examinez les données entrantes et l'invariant violé.

1.3 Les 3 Objectifs Sacrés de la Tâche de Sprint

Objectif	Composant Clé	Critère de Réussite Inviolable
1. Créer un Rôle	<code>creer_role()</code> + <code>Vue RoleListCreateView</code>	Code en majuscules, unique par tenant. Interdiction stricte de créer le rôle 'DG' ou 'ADMIN'. Transaction atomique.
2. Attribuer des Permissions	<code>RoleModulePermission</code> + Niveaux 0 à 3	Matrice complète sur les 12 modules opérationnels CCD Digital (AUCUN, LECTURE, ECRITURE, VALIDATION).
3. Rattacher un Collaborateur	<code>rattacher_collaborateur_a_role()</code> + Endpoint PATCH/POST	Affectation du rôle personnalisé à l'Utilisateur. Immutabilité du compte Propriétaire/DG. Synchronisation du rôle global.

SECTION 2 : ARCHITECTURE & INVARIANTS (PÓLYA / DEHAENE)

2.1 Modèle Mental & Invariants de George Pólya

Dans *How to Solve It*, George Pólya prescrit d'isoler en premier lieu l'Inconnue, les Données et la Condition. Dans notre architecture RBAC multi-tenant, les données et invariants sont formellement définis :

- **Invariant 1 (Isolation Tenant)** : La table role et la table utilisateur résident dans le schéma du tenant (ex: demo). Aucune requête ne doit mélanger les rôles de deux entreprises clientes distinctes.
- **Invariant 2 (Souveraineté du Propriétaire / DG)** : L'utilisateur fondateur avec is_owner=True a un statut et un rôle immuables. Aucune fonction ne peut réassigner son rôle ni désactiver son compte.
- **Invariant 3 (Atomicité de la Matrice)** : Un rôle ne peut jamais exister sans sa matrice de permissions. Si la création du rôle réussit mais que l'insertion d'un module échoue, le rollback de base de données doit être total (ACID via transaction.atomic()).
- **Invariant 4 (Règle des 12 Modules CCD Digital)** : Tout module renseigné doit appartenir rigoureusement à l'énumération ModuleChoix (CHANTIER, QHSE, ACHATS, STOCKS, FINANCE, RH, GED, MATERIEL, TIERS, CONTRATS, PILOTAGE, SUPPORT).
- **Invariant 5 (Échelle Ordinale des Droits)** : Les niveaux d'accès sont strictement ordonnés : 0 (AUCUN) < 1 (LECTURE) < 2 (ECRITURE) < 3 (VALIDATION).

2.2 Le Schéma Relationnel Entité-Association (MCD)

Fichier : Structure Relationnelle RBAC CCD Digital

Utilisateur		Role		RoleModulePermission
id: UUID (PK)	0..*	id: UUID (PK)	1	id: UUID (PK)
email: EmailField		code: CharField(50) [UNIQUE]		role_id: FK(Role)
role_global: CharField		libelle: CharField(100)	1..*	module: CharField(30)
role_personnalise_id		est_système: BooleanField		niveau: PositiveSmallInt
is_owner: BooleanField		est_actif: BooleanField		(0=AUCUN, 1=LEC, 2=ECR...)

2.3 Les 12 Modules Opérationnels BTP et leurs Niveaux

Le dictionnaire ci-dessous représente la matrice d'habilitation d'un rôle. Chaque clé correspond à un module BTP, et chaque valeur à son niveau de privilège.

Code Module	Description Métier BTP	Niveau Conseillé par Défaut
CHANTIER	Gestion des travaux, journal, avancement physique	2 (ECRITURE)
QHSE	Sécurité, incidents, EPI, conformité environnementale	1 (LECTURE)
ACHATS	Demandes d'achat, bons de commande chantiers	0 (AUCUN) ou 1
STOCKS	Matériaux sur site, réceptions, sorties magasin	1 (LECTURE)
FINANCE	Budgets, situations de paiement, décomptes	0 (AUCUN - Réservé DG/DF)
RH	Pointage ouvriers, présences journalières	1 (LECTURE)
GED	Plans, DOE, rapports d'expertise technique	1 (LECTURE)
MATERIEL	Engins, outillage lourd, maintenance préventive	1 (LECTURE)
TIERS	Fournisseurs, sous-traitants, bureaux de contrôle	1 (LECTURE)
CONTRATS	Marchés publics/privés, avenants, garanties	0 (AUCUN)
PILOTAGE	Tableaux de bord consolidés, ratios d'avancement	1 (LECTURE)
SUPPORT	Assistance technique et tickets plateforme	1 (LECTURE)

SECTION 3 : GUIDE D'IMPLÉMENTATION PAS-À-PAS POUR VOS MAINS

Consigne d'Or : Tapez chaque instruction manuellement dans votre éditeur (VS Code / Cursor). Ne faites aucun copier-coller. Observez l'autocomplétion de votre IDE, comprenez les imports et la logique métier.

3.1 Étape 1 : Programmer les Fonctions Métier dans la Couche Service

Ouvrez le fichier `apps/accounts/services/roles.py`. Vérifiez la présence des fonctions `creer_role` et `modifier_role`. Implémentez ensuite la nouvelle fonction fondamentale de rattachement : `rattacher_collaborateur_a_role()`.

Fichier : `apps/accounts/services/roles.py`

```
def rattacher_collaborateur_a_role(
    *,
    collaborateur: Utilisateur,
    role: Role | None = None,
    role_global: str | None = None,
    modifie_par: Utilisateur | None = None,
) -> Utilisateur:
    """Rattache un collaborateur à un rôle personnalisé ou met à jour son rôle global.

    Règles d'intégrité et de sécurité :
    - Le Propriétaire/Fondateur (is_owner=True) a un rôle immuable : interdiction de modification.
    - Le rôle de Directeur Général ne peut pas être attribué à un collaborateur standard.
    - Si un 'role' personnalisé est fourni, il doit être actif et non supprimé.
    - Seul le DG ou un Admin peut conférer le rôle ADMIN.
    """
    from django.core.exceptions import ValidationError
    from apps.core.enums import RoleGlobal, StatutUtilisateur

    # Règle 1 : Immutabilité du compte Propriétaire
    if collaborateur.is_owner:
        raise ValidationError(_("Le rôle et le statut du Propriétaire / Fondateur sont immuables."))

    # Règle 2 : Interdiction d'attribuer le rôle DG
    if role_global == RoleGlobal.DIRECTEUR_GENERAL:
        raise ValidationError(_("Le rôle de Directeur Général est unique et ne peut pas être attribué."))

    # Règle 3 : Validation de l'existence et de l'état du rôle personnalisé
    if role is not None:
        if role.supprime_le is not None or not role.est_actif:
            raise ValidationError(_("Le rôle spécifié est inactif ou a été supprimé."))

    with transaction.atomic():
        champs_a_mettre_a_jour = ["modifie_le"]

        if role is not None:
            collaborateur.role_personnalise = role
            champs_a_mettre_a_jour.append("role_personnalise")
            # Si le code du rôle personnalisé correspond à un rôle global connu, synchroniser
            if role.code in RoleGlobal.values:
                collaborateur.role_global = role.code
                champs_a_mettre_a_jour.append("role_global")

        if role_global is not None and role_global in RoleGlobal.values:
            collaborateur.role_global = role_global
            if "role_global" not in champs_a_mettre_a_jour:
                champs_a_mettre_a_jour.append("role_global")

        collaborateur.save(update_fields=champs_a_mettre_a_jour)

    return collaborateur
```

Éclairage d'Ingénierie ORM Django (Melé 2024)

Pourquoi `update_fields=['...']` ?

En Django, faire un `save()` complet réécrit toutes les colonnes SQL. Si un autre processus a modifié `tentatives_echouees` ou `derniere_connexion` en parallèle, un `save` global écraserait ces valeurs (race condition). Utiliser `update_fields` génère une requête SQL `UPDATE utilisateur SET role_personnalise_id=..., modifie_le=... WHERE id=...` chirurgicale et infalsifiable.

3.2 Étape 2 : Définir les Contrats de Validation (Sérialiseurs DRF)

Le sérialiseur est le premier rempart du système. Il valide les types entrants, vérifie les contraintes métier et garantit qu'aucune donnée malformée n'atteint l'ORM. Ouvrez `apps/accounts/serializers/collaborateur.py` et ajoutez le sérialiseur de rattachement de rôle :

Fichier : `apps/accounts/serializers/collaborateur.py`

```
class CollaborateurRattacherRoleSerializer(serializers.Serializer):
    """Sérialiseur pour la mise à jour ou le rattachement de rôle d'un collaborateur existant."""

    role_global = serializers.ChoiceField(choices=RoleGlobal.choices, required=False)
    role_personnalise_id = serializers.UUIDField(required=False, allow_null=True)

    def validate(self, attrs):
        role_global = attrs.get("role_global")
        role_id = attrs.get("role_personnalise_id")

        if role_global is None and role_id is None:
            raise serializers.ValidationError(
                _("Au moins un rôle global ou un identifiant de rôle personnalisé doit être spécifié.")
            )

        if role_global == RoleGlobal.DIRECTEUR_GENERAL:
            raise serializers.ValidationError(
                {"role_global": _("Le rôle de Directeur Général ne peut pas être attribué.")}
            )

        if role_id is not None:
            role = Role.objects.filter(id=role_id, supprime_le__isnull=True).first()
            if not role:
                raise serializers.ValidationError(
                    {"role_personnalise_id": _("Rôle personnalisé introuvable ou inactif.")}
                )
            attrs["role_instance"] = role

        return attrs
```

3.3 Étape 3 : Créer la Vue d'API DRF Dédiée

Ouvrez `apps/accounts/views/collaborateur.py`. Vous allez créer la vue de détail `ParametresCollaborateurDetailView` permettant de modifier un collaborateur et de rattacher son rôle via des requêtes PATCH et POST :

Fichier : apps/accounts/views/collaborateur.py

```

class ParametresCollaborateurDetailView(APIView):
    """`GET`, `PATCH` et `POST` /api/v1/parametres/collaborateurs/{id}/`.
    Permet de consulter le détail d'un collaborateur et de mettre à jour son rôle.
    """

    permission_classes = [IsAuthenticated]
    parser_classes = [JSONParser]

    @extend_schema(
        summary="Rattacher un rôle à un collaborateur ou modifier ses informations",
        request=CollaborateurRattacherRoleSerializer,
        responses={200: CollaborateurResponseSerializer},
    )
    def patch(self, request, pk):
        _autoriser_parametres_collaborateurs(request.user)
        collaborateur = get_object_or_404(Utilisateur, pk=pk, supprime_le__isnull=True)

        serializer = CollaborateurRattacherRoleSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        role_instance = serializer.validated_data.get("role_instance")
        role_global = serializer.validated_data.get("role_global")

        # Règle R-DEMO-01 : Seul le DG ou Propriétaire peut attribuer le rôle ADMIN
        if role_global == RoleGlobal.ADMIN and not (
            getattr(request.user, "is_dg", False) or getattr(request.user, "is_owner", False)
        ):
            raise ActionInterditeDelegue()

        try:
            collaborateur = rattacher_collaborateur_a_role(
                collaborateur=collaborateur,
                role=role_instance,
                role_global=role_global,
                modifie_par=request.user,
            )
        except DjangoValidationError as exc:
            msg = str(exc.message if hasattr(exc, "message") else exc)
            raise ValidationError({"detail": msg}) from exc

        # Recharger les données complètes pour la réponse
        rp_data = None
        if collaborateur.role_personnalise:
            rp_data = {
                "id": collaborateur.role_personnalise.id,
                "code": collaborateur.role_personnalise.code,
                "libelle": collaborateur.role_personnalise.libelle,
            }
        reponse_data = {
            "id": collaborateur.id,
            "email": collaborateur.email,
            "nom": collaborateur.nom,
            "prenom": collaborateur.prenom,
            "nom_complet": collaborateur.nom_complet,
            "telephone": collaborateur.telephone,
            "role_global": collaborateur.role_global,
            "role_global_libelle": collaborateur.get_role_global_display(),
            "role_personnalise": rp_data,
            "statut": collaborateur.statut,
            "is_owner": collaborateur.is_owner,
            "cree_le": collaborateur.cree_le,
            "projets": [],
            "lien_activation": None,
        }
        return Response(CollaborateurResponseSerializer(reponse_data).data, status=status.HTTP_200_OK)

    def post(self, request, pk):
        return self.patch(request, pk)

```

3.4 Étape 4 : Déclarer la Route URL dans l'Application

Ouvrez `apps/accounts/urls.py` et enregistrez la route REST pour accéder au collaborateur par son identifiant UUID :

Fichier : `apps/accounts/urls.py`

```
# Collaborateurs – Paramètres (/api/v1/parametres/collaborateurs/)
path(
    "parametres/collaborateurs/",
    ParametresCollaborateurListCreateView.as_view(),
    name="parametres-collaborateurs-liste-creer",
),
path(
    "parametres/collaborateurs/<uuid:pk>/",
    ParametresCollaborateurDetailView.as_view(),
    name="parametres-collaborateur-detail-modifier",
),
```

SECTION 4 : SIGNAL D'ERREUR BAYÉSIEN & PRE-MORTEM (KAHNEMAN)

Dans la technique du **Pre-Mortem** (Gary Klein & Daniel Kahneman), nous nous projetons dans le futur où notre code est en production et s'est effondré. Nous analysons rétrospectivement ce qui a causé la panne afin de blinder le code dès maintenant :

Scénario de Panne Potentiel	Mécanisme de Défaillance	Parade Préventive Programmée
1. Usurpation de Droits Admin	Un administrateur délégué s'auto-attribue des droits ou élève un complice au rôle DG.	Contrôle strict dans <code>rattacher_collaborateur_a_role</code> : le rôle DG est non attribuable et seul le DG peut nommer un ADMIN.
2. Rôle Fantôme Supprimé	Un collaborateur est rattaché à un rôle qui a subi une suppression logique (<code>supprime_le</code> is not None).	Le serializer vérifie <code>supprime_le__isnull=True</code> et <code>est_actif=True</code> avant d'accepter le rattachement.
3. Écrasement Imprévu (Race Condition)	Deux requêtes concurrentes modifient l'utilisateur en même temps.	Utilisation systématique de <code>save(update_fields=[...])</code> et transaction atomique with <code>transaction.atomic()</code>.
4. Rétrocompatibilité Rôle Global	Les anciens modules qui lisent <code>user.role_global</code> au lieu de <code>role_personnalise</code> ne voient pas le changement.	Synchronisation automatique bidirectionnelle : si le code du rôle personnalisé correspond à un rôle global, <code>role_global</code> est mis à jour.
5. Violation de Schéma Multi-Tenant	Une requête sur les rôles s'exécute dans le schéma public au lieu du tenant.	L'ORM est encapsulé dans le middleware <code>TenantMainMiddleware</code>. Les tests utilisent <code>schema_context(SCHEMA)</code>.

SECTION 5 : CHECKLIST DE TESTS & VALIDATION MANUELLE

Pour ancrer vos acquis (Pilier 3 de Dehaene : Retour sur Erreur Actif), vous allez rédiger et exécuter la suite de tests unitaires et d'intégration automatisés.

Fichier : `apps/accounts/tests/test_parametres_collaborateurs.py`

```
@pytest.mark.django_db
def test_rattacher_collaborateur_a_role_personnalise(client_tenant, admin_user, collaborateur_user):
    """Vérifie qu'un administrateur peut rattacher un collaborateur à un rôle personnalisé."""
    with schema_context(SCHEMA):
        role_maitre = Role.objects.create(
            code="CHEF_EQUIPE_COFFRAGE",
            libelle="Chef d'Équipe Coffrage",
            est_système=False,
            est_actif=True,
        )

    client = _auth(client_tenant, admin_user)
    payload = {"role_personnalise_id": str(role_maitre.id)}

    rep = client.patch(f"/api/v1/parametres/collaborateurs/{collaborateur_user.id}/", payload, format="json")
    assert rep.status_code == status.HTTP_200_OK
    data = rep.json()
    assert data["role_personnalise"]["code"] == "CHEF_EQUIPE_COFFRAGE"

    with schema_context(SCHEMA):
        collaborateur_user.refresh_from_db()
        assert collaborateur_user.role_personnalise_id == role_maitre.id

@pytest.mark.django_db
def test_interdiction_modifier_role_proprietaire(client_tenant, admin_user, dg_user):
    """Rejette toute tentative de modification du rôle du Propriétaire / DG."""
    client = _auth(client_tenant, admin_user)
    payload = {"role_global": RoleGlobal.CHEF_CHANTIER}

    rep = client.patch(f"/api/v1/parametres/collaborateurs/{dg_user.id}/", payload, format="json")
    assert rep.status_code == status.HTTP_400_BAD_REQUEST
    assert "immuable" in str(rep.json()).lower()
```

5.2 Protocole de Validation en Ligne de Commande

Commandes Terminal de Vérification

1. Exécuter l'intégralité des tests de rôles et collaborateurs :

```
pytest apps/accounts/tests/test_parametres_roles.py apps/accounts/tests/test_parametres_collaborateurs.py -v
```

2. Tester avec une requête cURL réaliste via l'hôte tenant local :

```
curl -X PATCH http://demo.localhost:8000/api/v1/parametres/collaborateurs/<UUID>/ \
-H 'Host: demo.localhost' \
-H 'Authorization: Bearer <TOKEN_JWT>' \
-H 'Content-Type: application/json' \
-d '{"role_personnalise_id": "<ROLE_UUID>"}
```

SECTION 6 : DÉFI HOMO DOCENS & CONSOLIDATION NOCTURNE

6.1 Le Défi de l'Homo Docens (Enseigner pour Maîtriser)

L'expression latine *Homo Docens* désigne l'homme qui enseigne. En sciences cognitives (Principe 8 de Lovett et al.), le niveau le plus élevé de maîtrise métacognitive est atteint lorsque vous êtes capable d'expliquer l'architecture à un pair. Avant de considérer cette tâche terminée, prenez une feuille blanche ou ouvrez un bloc-notes et répondez aux 3 questions suivantes sans regarder le code :

- **Question 1** : Pourquoi est-il indispensable d'utiliser `transaction.atomic()` lors de la création d'un rôle personnalisé avec sa matrice de permissions ?
- **Question 2** : Quelle est la différence fondamentale entre `user.role_global` et `user.role_personnalise`, et comment garantit-on la rétrocompatibilité des anciens endpoints ?
- **Question 3** : Comment le système empêche-t-il qu'un administrateur délégué ne révoque le compte du Directeur Général créateur du tenant ?

6.2 Rituel de Somnolence & Replay Neuronal Nocturne (Murphy / Dehaene)

Stanislas Dehaene (Pilier 4 : Consolidation) et le Dr. Joseph Murphy soulignent le rôle déterminant du sommeil dans l'ancrage définitif des apprentissages. Pendant le sommeil lent profond, l'hippocampe 'rejoue' les circuits neuronaux stimulés pendant la journée à une vitesse 20 fois supérieure à la normale, transférant les connaissances vers le néocortex durable.

Protocole d'Incubation & Consolidation

Rituel du Soir avant l'Endormissement :

1. **Déconnectez vos yeux des écrans** 30 minutes avant de dormir.
2. **Visualisez mentalement** le flux de données : de la requête HTTP entrante jusqu'à l'écriture PostgreSQL, en passant par la validation du sérialiseur et l'isolation du schéma tenant.
3. **Remerciez votre subconscient** pour le travail accompli : « *Tout ce que j'ai implémenté aujourd'hui est parfaitement assimilé, ordonné et gravé dans ma mémoire d'ingénieur. Demain, j'aborderai la suite du Sprint avec aisance et souveraineté.* »

Validé par le Développeur :

Nom : _____

Date : ____ / ____ / 2026

Signature :

Approuvé par le Mentor Neurocognitif :

Agentic AI Pair Programmer & Cognitive Architect

Certification : CCD Digital Enterprise Grade

Statut : Conforme Sprint 3 LifeCycle